

Paradigmas de Programación

Trabajo Práctico N° 2

El presente trabajo práctico tiene como objetivo aplicar los conceptos, principios y características del paradigma de la programación orientada a objetos (Clases, objetos, métodos, interfaz, herencia, polimorfismo, colecciones, etc).

Fecha de entrega: 30 de Junio / 03 de Julio de 2026 (dependiendo la comisión)

Objetivo

Implementar los conceptos abordados del paradigma de la Programación orientada a objetos, como solución a un problema real, partiendo del análisis de la problemática, el diseño de la solución (Diagrama de Clases) y posteriormente la construcción del producto final (Software).

Consigna

Temática

Batalla de Magos contra Mortífagos

Objetivos del trabajo

- Modelar un sistema de combate entre magos y mortífagos
- Implementar hechizos y habilidades especiales
- Gestionar batallones de magos y mortífagos, utilizando distintos tipos de colección y aplicando los patrones de diseño que correspondan
- Usar herencia y polimorfismo para permitir la diversidad de magos (aurors, estudiantes, profesores) y mortífagos (seguidores comunes, comandantes, etc.).

Descripción General

El trabajo simulará una serie de batallas entre dos facciones: los magos y los mortífagos. Cada facción tendrá diferentes tipos de personajes, con sus propios hechizos, habilidades y características. Se espera que el sistema esté diseñado de manera flexible para que se puedan agregar nuevos tipos de personajes y hechizos fácilmente.

1. Clases Base

Personaje (abstracta): Tendrá atributos como nombre, nivel de magia, puntos de vida, y una lista de hechizos que puede lanzar.

Mago (hereda de Personaje): Se especializa en hechizos defensivos y de ataque. Subclases: Auror, Estudiante, Profesor.

Mortífago (hereda de Personaje): Se enfoca en ataques oscuros y habilidades especiales. Subclases: Seguidor, Comandante.

2. Hechizos y Habilidades

Implementar diferentes tipos de hechizos. El diseño debe permitir agregar un hechizo nuevo sin modificar los existentes ni el resto del sistema. Cada hechizo encapsula su propio comportamiento

Hechizo (interface): Tendrá un método ejecutar, que aplicará un efecto a un personaje (daño, curación, efecto de estado). Tipos de Hechizo: "Expelliarmus", "Avada Kedavra", "Protego", etc.

Cada hechizo se implementa de forma diferente. Por ejemplo, "Expelliarmus" hace daño, "Protego" protege a quien lo lanza y "Expecto Patronum" cura. Parte del trabajo es decidir qué estructura de diseño conviene para que cada hechizo defina su propio efecto.

3. Creación de personajes y hechizos

Centralizar y ocultar la creación de personajes y hechizos, de modo que pedir "un mago" o "un hechizo de ataque" no obligue a conocer los detalles de su construcción

Por ejemplo, un componente encargado de armar magos o mortífagos con sus habilidades iniciales ya asignadas. Y otro encargado de crear hechizos según un tipo o un nivel de dificultad.

4. Gestión de Batallones

Usar distintos tipos de colección para manejar los equipos, eligiendo cada uno según sus propiedades. Sugerencias:

- List para la secuencia de acciones.
- Map para registrar los hechizos lanzados por cada personaje.
- Set para evitar que un batallón repita un mismo hechizo dentro de su turno.

5. Polimorfismo y Herencia

Crear una jerarquía de personajes con atributos y métodos comunes en la clase base Personaje, y comportamientos diferenciados en las subclases. La diferencia entre tipos debe surgir del polimorfismo, no de preguntar el tipo con if o instanceof. Por ejemplo, un mismo hechizo puede causar más o menos daño según quién lo lanza (un mortífago es más letal con hechizos oscuros, un mago cura mejor), sin que el hechizo necesite conocer el tipo del lanzador.

6. Reglas de la Batalla

La batalla se desarrolla en rondas. En cada ronda, los batallones juegan por turnos: en el turno de un batallón, cada uno de sus integrantes en pie realiza una acción.

La acción puede ser lanzar un hechizo (de ataque, de defensa como Protego, o de curación) o usar un objeto mágico (ver Bonus 1). Dentro de un mismo turno, un batallón no puede repetir un hechizo.

Cuando los puntos de vida de un personaje llegan a cero, queda eliminado del combate: no puede atacar ni ser objetivo

7. Pruebas

Crear una batería de pruebas unitarias para los hechizos, personajes y la lógica de las batallas.

Utilizar JUnit para esto, y validar que los hechizos se ejecuten correctamente, que los personajes pierdan vida según el hechizo recibido, y que las colecciones de personajes y hechizos funcionen adecuadamente.

8. Bonus

Las siguientes funcionalidades son adicionales y tienen una dificultad similar. Debe implementarse al menos una; quien quiera puede sumar más. Conviene elegir la que resulte más interesante.

Objetos mágicos: un personaje puede equipar objetos que modifican sus capacidades mientras los lleve (una varita que potencia ciertos hechizos, una capa que esquiva el próximo ataque, un amuleto que recupera más magia al descansar). El mismo personaje, con o sin objetos, se comporta distinto sin modificar su clase. Los objetos se pueden equipar y quitar durante la batalla.

Personaje reactivo: que un personaje reaccione automáticamente a lo que ocurre en la batalla. Por ejemplo, un Comandante que, al caer un aliado de su batallón, entre en un estado especial que potencie su próximo ataque y se consuma al usarlo. El cambio de comportamiento debe verse en la salida.

Hechizos con efectos prolongados: hechizos que dejan al objetivo en un estado temporal por varios turnos. Por ejemplo, un sangrado que aplica daño al inicio de cada uno de los próximos turnos del objetivo, una paralización que le impide actuar, o una regeneración que le devuelve PV. El diseño debe permitir agregar nuevos estados sin tocar los existentes y que varios convivan en un mismo personaje.

Interfaz

Se evaluará el correcto funcionamiento del sistema utilizando dos enfoques simultáneos:

1. Se deberá probar cada una de las funcionalidades desarrolladas, utilizando JUnit correctamente
2. Se podrá ejecutar un main, en el cual se creen los batallones y se los haga combatir en forma automática.

Ejemplo de main (no exhaustivo):

```
public class BatallaMagosVsMortifagos {  
    public static void main(String[] args) {  
        Batallon batallonMagos = new Batallon();  
        Batallon batallonMortifagos = new Batallon();  
        for (int i = 0; i < 3; i++) {  
            batallonMagos.agregarPersonaje(Reclutador.crearMago());  
            batallonMortifagos.agregarPersonaje(Reclutador.crearMortifago());  
        }  
        Random rand = new Random();  
        while (batallonMagos.tienePersonajesSaludables() && batallonMortifagos.tienePersonajesSaludables()) {  
            // los ataques pueden tener salidas por pantalla para mostrar lo que sucede
```

```

if (rand.nextBoolean()) {
    batallonMagos.atacar(batallonMortifagos);
    if (batallonMortifagos.tienePersonajesSaludables()) {
        batallonMortifagos.atacar(batallonMagos);
    }
} else {
    batallonMortifagos.atacar(batallonMagos);
    if (batallonMagos.tienePersonajesSaludables()) {
        batallonMagos.atacar(batallonMortifagos);
    }
}

System.out.println("-----");
}

if (batallonMagos.tienePersonajesSaludables()) {
    System.out.println("¡Los magos han ganado la batalla!");
} else {
    System.out.println("¡Los mortífagos han ganado la batalla!");
}
}

```

Sugerimos tener varios escenarios preparados, para mostrar todas las características desarrolladas.

Entrega

El trabajo se realizará en forma grupal, con equipos de 4 a 6 personas.

Se espera un informe que contenga al menos: carátula, índice, introducción, desarrollo, conclusiones y referencias (APA).

Habrà una defensa y presentación oral en la fecha designada.